The History of Software

The first theory about software was proposed by Alan Turing in his 1935 essay *Computable numbers with an application to the Entscheidungsproblem (Decision problem)*. The term "software" was first used in print by John W. Tukey in 1958. Colloquially, the term is often used to mean application software. **In computer science and software engineering, software is all information processed by computer system, programs and data**. The academic fields studying software are computer science and software engineering. From its beginnings in the 1940s, writing software has evolved into a profession concerned with how best to maximize the quality of software and of how to create it. **Quality can refer to how maintainable software is, to its stability, speed, usability, testability, readability, size, cost, security, and number of flaws or "bugs", as well as to less measurable qualities like elegance, conciseness, and customer satisfaction, among many other attributes.** How best to create high quality software is a separate and controversial problem covering software design principles, so-called "best practices" for writing code, as well as broader management issues such as optimal team size, process, how best to deliver software on time and as quickly as possible, work-place "culture," hiring practices, and so forth. All this falls under the broad rubric of software engineering.

1945-1965: The Beginning

The term *software engineering* first appeared in the late 1950s and early 1960s. Programmers have always known about civil, electrical, and computer engineering and debated what engineering might mean for software.

The NATO Science Committee sponsored two conferences on software engineering in 1968 and 1969, which gave the field its initial boost. Many believe these conferences marked the official start of the profession of software engineering.

1968: NATO Software Engineering Conference, Garmisch, Germany.
1969: NATO Conference on Software Engineering, Rome, Italy.

1965-1985: The Software Crisis

Software engineering was spurred by the so-called "software crisis" of the 1960s, 1970s, and 1980s, which identified many of the problems of software development. Many software projects ran over budget and schedule. Some projects caused property damage. A few projects caused loss of life. The software crisis was originally defined in terms of productivity, but evolved to emphasize quality. Some used the term *software crisis* to refer to their inability to hire enough qualified programmers.

- **Cost and Budget Overruns**: The OS/360 operating system was a classic example. This decade-long project from the 1960s eventually produced one of the most complex software systems at the time. OS/360 was one of the first large (1000 programmers) software projects. Fred Brooks claims in *The Mythical Man Month* that he made a multi-million dollar mistake of not developing a coherent architecture before starting development.
- **Property Damage:** Software defects can cause property damage. Poor software security allows hackers to steal identities, costing time, money, and reputations.
- **Life and Death**: Software defects can kill. Some embedded systems used in radiotherapy machines failed so catastrophically that they administered lethal doses of radiation to patients. The most famous of these failures is the //Therac 25// incident.

1985-1989: Silver Bullets

For decades, solving the software crisis was paramount to researchers and companies producing software tools. Seemingly, they trumpeted every new technology and practice from the 1970s to the 1990s as a "silver bullet" to solve the software crisis. Tools, discipline, formal methods, process, and professionalism were touted as silver bullets:

- **Tools**: Especially emphasized were tools: structured programming, object-oriented programming, CASE tools (set **of software application programs**, which are used to automate SDLC activities. CASE tools are used by software project managers, analysts

and engineers to develop software system), Ada, documentation, and standards were touted as silver bullets.

- **Discipline**: Some pundits argued that the software crisis was due to the lack of discipline of programmers.
- **Formal methods**: Some believed that if formal engineering methodologies would be applied to software development, then production of software would become as predictable an industry as other branches of engineering. They advocated proving all programs correct.
- **Process**: Many advocated the use of defined processes and methodologies like the Capability Maturity Model.
- **Professionalism**: This led to work on a code of ethics, licenses, and professionalism.

In 1986, Fred Brooks published his *No Silver Bullet* article, arguing that no individual technology or practice would ever make a 10-fold improvement in productivity within 10 years. Debate about silver bullets raged over the following decade. Advocates for Ada, components, and processes continued arguing for years that their favorite technology would be a silver bullet. Skeptics disagreed. Eventually, almost everyone accepted that no silver bullet would ever be found. Yet, claims about *silver bullets* pop up now and again, even today.

1990-1999: The Internet

The rise of the Internet led to very rapid growth in the demand for international information display/e-mail systems on the World Wide Web. Programmers were required to handle illustrations, maps, photographs, and other images, plus simple animation, at a rate never before seen, with few well-known methods to optimize image display/storage (such as the use of thumbnail images).

The growth of browser usage, running on the HTML language, changed the way in which information-display and retrieval was organized. The widespread network connections led to the growth and prevention of international computer viruses on MS Windows computers, and the vast proliferation of spam e-mail became a major design issue in e-mail systems, flooding communication channels and requiring semi-automated pre-screening. Keyword-search systems

evolved into web-based search engines, and many software systems had to be re-designed, for international searching, depending on search engine optimization (SEO) techniques. Human natural-language translation systems were needed to attempt to translate the information flow in multiple foreign languages, with many software systems being designed for multi-language usage, based on design concepts from human translators. Typical computer-user bases went from hundreds, or thousands of users, to, often, many-millions of international users.

2000-Present: Lightweight Methodologies

With the expanding demand for software in many smaller organizations, the need for inexpensive software solutions led to the growth of simpler, faster methodologies that developed running software, from requirements to deployment, quicker & easier. The use of rapid-prototyping evolved to entire *lightweight methodologies*, such as Extreme Programming (XP), which attempted to simplify many areas of software engineering, including requirements gathering and reliability testing for the growing, vast number of small software systems. Very large software systems still used heavily-documented methodologies, with many volumes in the documentation set; however, smaller systems had a simpler, faster alternative approach to managing the development and maintenance of software calculations and algorithms, information storage/retrieval and display.

Current Trends in Software Engineering

Software engineering is a young discipline, and is still developing. The directions in which software engineering is developing include:

Aspects

Aspects help software engineer's deal with quality attributes by providing tools to add or remove boilerplate code from many areas in the source code. Aspects describe how all objects or functions should behave in particular circumstances. For example, aspects can add debugging, logging, or locking control into all objects of particular types. Researchers are currently working to understand how to use aspects to design general-purpose code. Related concepts include generative programming and templates.

Agile

Agile software development guides software development projects that evolve rapidly with changing expectations and competitive markets. Proponents of this method believe that heavy, document-driven processes (like TickIT, CMM and ISO 9000) are fading in importance. Some people believe that companies and agencies export many of the jobs that can be guided by heavy-weight processes. Related concepts include extreme programming, scrum, and lean software development.

Experimental

Experimental software engineering is a branch of software engineering interested in devising experiments on software, in collecting data from the experiments, and in devising laws and theories from this data. Proponents of this method advocate that the nature of software is such that we can advance the knowledge on software through experiments only.

Model-driven

Model driven design develops textual and graphical models as primary design artifacts. Development tools are available that use model transformation and code generation to generate well-organized code fragments that serve as a basis for producing complete applications.